

Beyond feature parity

What a backend-neutral regression language makes possible

BayesianRegressionModels.jl collaborator deck

2026-09-13

This is not a feature race

THE COMPARISON THAT MATTERS

brms already owns the mature applied-regression benchmark. The question is what each architecture makes *cheap to add next*.

BRMS TODAY

Deep, coherent Stan generation

Distributional, nonlinear, multivariate, smooth, autocorrelated, censored, missing-data, and custom-family models—with a rich post-processing ecosystem.

BRM TODAY

One formula, shared semantics

The same **BRMI** is prepared once, then lowered to generated StanBlocks/Stan or a direct Turing model.

THE FUTURE AXIS

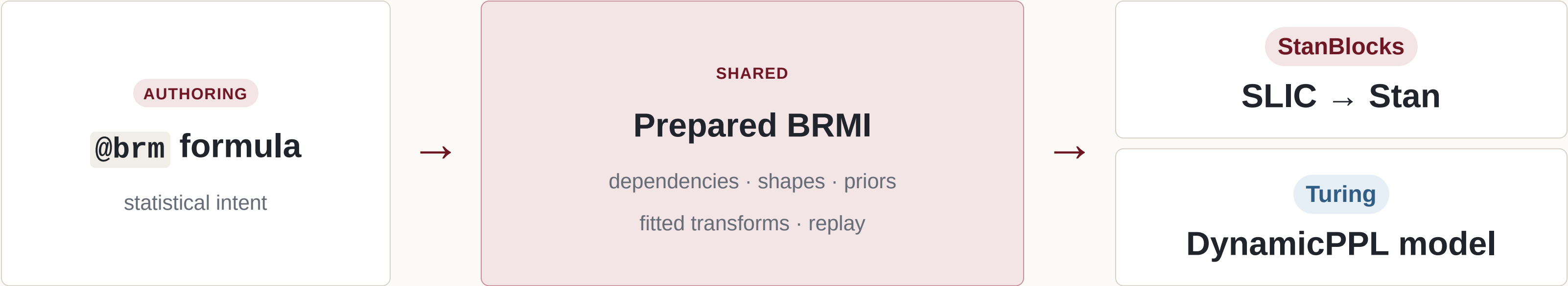
Where can computation vary without rewriting statistical intent?

That is an architectural capability—not a claim that every implementation already exists.

One declaration; two computational realizations

DEMONSTRATED ARCHITECTURE

```
@brm begin · μ ~ 1 + treatment + hsgp(time) + (1 + treatment | site) · log(σ) ~ 1 + hsgp(time) · y ~  
censored(Normal(μ, σ), lower, upper) · end
```



The invariant is the prepared statistical program. Parameter geometry and execution belong to the backend.

1 · Extension can mean composition, not surgery

FUTURE CAPABILITY AXIS		
	BRMS	BRM.JL
Custom response	Excellent supported route: <code>custom_family()</code> + Stan functions + R post-processing hooks.	Callable Julia distribution/factory survives preparation; Turing can execute it directly. StanBlocks adds an explicit translation.
New term	Extend formula/parser/code generation, or inject Stan with <code>stanvar()</code> .	Add a typed prepared record; implement backend methods only where execution exists.
Composition	Large built-in grammar; custom pieces must integrate with generated Stan and downstream methods.	Prepared callables, evidence wrappers, weights, links, and terms compose before backend lowering.

FAIRNESS

brms already exposes unusually strong escape hatches. Its constraint is deliberate: the executable language is Stan.

ARCHITECTURE-ENABLED

BRM can make “native Julia first, Stan translation second” a normal extension path—but only after validation, replay, and output contracts are implemented.

2 · Structure need not fit one rectangular table

FUTURE CAPABILITY AXIS

BRMS STRENGTH

Nonlinear formulas of arbitrary complexity, multivariate models, autocorrelation, splines/GPs, phylogenetic covariance, and custom Stan give broad structural reach.

Its primary user contract remains a formula plus a `data.frame`, with extra data/code supplied through established escape hatches.

BRM DEMONSTRATED

Different axes can meet inside one model.

$$C_j(t) = \frac{D_j}{V_j} \exp\left(-\frac{CL_j}{V_j}t\right), \quad y_{jk} \sim \mathcal{N}(C_j(t_{jk}), \sigma)$$

`kernel(...)` evaluates one structural cell per subject;
`ragged(time, subject)` joins an observation axis; formula predictors still own population and group effects.

ENABLED WITH ENGINEERING State-space, ODE, renewal, and multistream models can share formula-level priors/evidence while backend-native cells own mechanics. Each new cell still needs explicit shape, AD, replay, and output support.

3 · A backend can change the question—not the model

FUTURE CAPABILITY AXIS

StanBlocks only

Generated program control

Stan compilation, BridgeStan density access, SLIC extension hooks, descriptor-driven draw transport, and current online WarmupHMC adaptive-centering machinery.

Turing only

Julia-native execution

Untranslated Julia distributions and functions, DynamicPPL model composition, and direct access to Turing's inference ecosystem.

Both

Shared statistical contract

Population/distributional predictors, broad priors and likelihoods, group effects, advanced terms, fitted preprocessing, pointwise likelihood, prediction, and replay—within stated subsets.

Future bet: choose marginalization, HMC implementation, automatic differentiation, particles, or specialized kernels *after* model preparation—not by forking the public formula language.

4 · Geometry is a model implementation choice

WHY CENTERING BELONGS ABOVE HANDWRITTEN CODE

CENTERED

$$\beta_k \sim \mathcal{N}(0, s_k)$$

The physical coefficient is sampled directly.

Often better when the likelihood strongly identifies the coefficient.

NONCENTERED

$$z_k \sim \mathcal{N}(0, 1), \quad \beta_k = s_k z_k$$

Prior scale and latent coordinate are separated.

Often better when data are weak and funnel geometry dominates.

THE HARD PART

The preferred coordinate can differ by group, term, frequency, and stage of adaptation. A fixed global switch throws information away.

HSGP turns one geometry choice into many

ADAPTIVE-CENTERING CASE STUDY · INTEGRATION GATE

$$f(x) \approx \sum_{k=1}^m \phi_k(x) s_k(\rho, \sigma) z_k$$

Low-frequency and high-frequency basis weights can have very different spectral scales and posterior information.

With partial centeredness $c_k \in [0, 1]$:

$$u_k \sim \mathcal{N}(0, s_k^{2c_k}), \quad \beta_k = s_k^{1-c_k} u_k$$

The physical function is unchanged; only its coordinates move.

Case artifact is intentionally gated

The dependency lane is still validating the immutable Generable motorcycle reproduction, backend parity, diagnostics, and plots.

Final deck will replace this panel only after that work lands canonically.

Same posterior target; measurable geometry

ADAPTIVE-CENTERING CASE STUDY · INTEGRATION GATE

NONCENTERED

All $c_k = 0$.

Pilot baseline and a robust weak-data default.

CENTERED

All $c_k = 1$.

Useful control; can amplify scale–weight coupling.

ADAPTIVE

Choose each c_k from pilot draws, then refit with a fixed geometry.

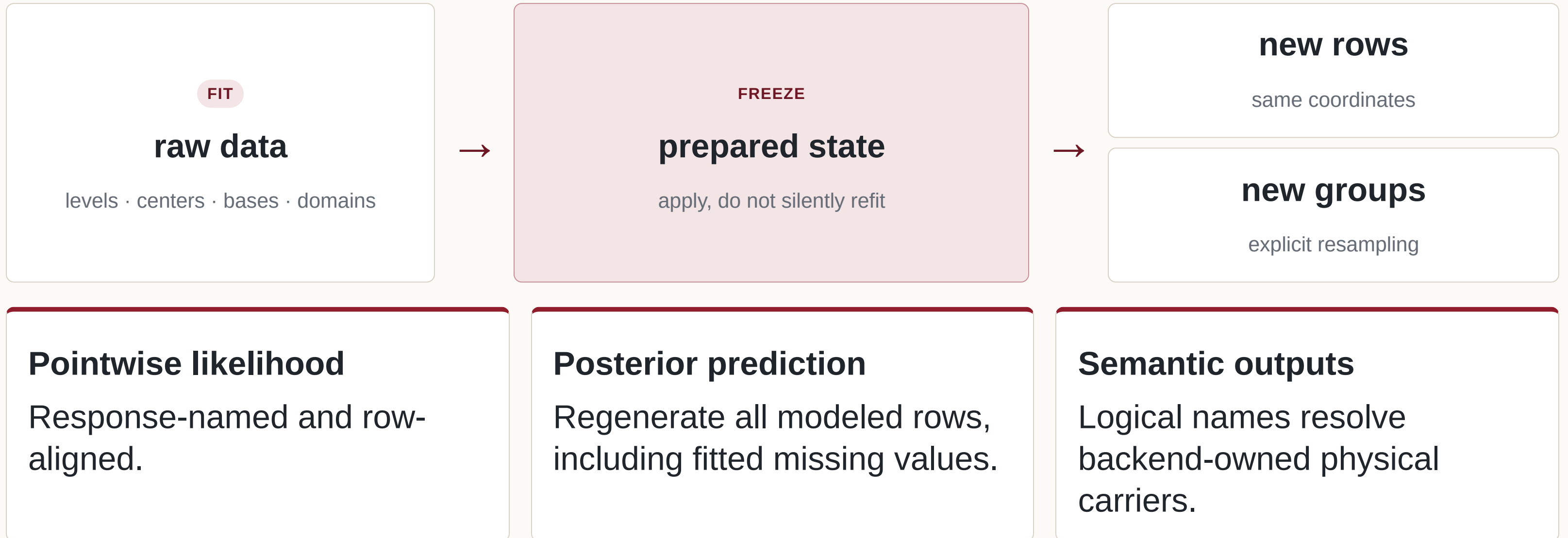
Offline pilot/refit—not online adaptation.

Awaiting reproducible results panel

Selected centeredness · posterior mean/scale bands · R-hat/ESS/divergences · gradient evaluations · wall time · both backends

5 · Replay and outputs are part of model semantics

FUTURE CAPABILITY AXIS



The horizon: evidence before promise

CAPABILITY MAP

	DEMONSTRATED NOW	ENABLED WITH ENGINEERING	OPEN RESEARCH QUESTION
Composition	Callable priors/likelihoods, modifiers, typed terms	One extension package carrying shared term semantics across backends	Can capability inference replace family/term admission tables entirely?
Structure	Multi-axis kernels; PK, renewal, IPM examples	Reusable ODE/state-space cells with standard replay/output traits	How much structure belongs in formula syntax versus typed functions?
Backends	Generated Stan + direct Turing from one BRMI	Backend-specific marginalization, AD, sampler, and kernel choices	Which semantic equivalence tests are sufficient across executors?
Geometry	Centered/noncentered blocks; adaptive HSGP candidate	Shared geometry metadata and diagnostics for more term classes	Can warmup select parameterization without invalid adaptation?
Replay	Frozen transforms, explicit new-group semantics	One descriptor/output API across Stan and Julia executors	How should stored fits survive backend evolution?

The bet

END OF MAIN TALK

A regression DSL becomes a research platform when the statistical program survives changes in backend, geometry, and execution strategy.

brms shows how far a coherent high-level language can take applied Bayesian modeling. BRM's distinct wager is to preserve that coherence while making the computational substrate plural.

DEMONSTRATED

ENGINEERING PATH

RESEARCH AGENDA

Current parity, compressed

COLLABORATOR APPENDIX A

SURFACE	STATUS	BOUNDARY WORTH REMEMBERING
Population & distributional predictors	Both	Shared designs, links, coefficient addressing, callable scalar priors.
Group effects	Both · partial	Plain/correlated/ /shared-ID/stratified/mm; adaptive geometry is not yet shared.
Advanced terms	Both · subsets	s , t2 , me , mo , AR/DAR, GP/HSGP, structured fields; custom SLIC needs a Julia method.
Responses & evidence	Both · subsets	Multiple/joint/ordinal, weights, missingness, truncation/censoring; some combinations fail closed.
Replay & prediction	Both · subsets	Frozen transforms and explicit group resampling; output surfaces are not yet identical.
Adaptive geometry / descriptor transport	StanBlocks lead	WarmupHMC integration, executable descriptor, unconstrained-draw transport.
Untranslated Julia callable	Turing only	StanBlocks requires an explicit Stan translation and executable companion paths.

What “shared preparation” contains

COLLABORATOR APPENDIX B

PROGRAM

- dependency-ordered operations
- sampled declarations and deterministic assignments
- population predictors and term records
- observations, modifiers, weights, row axes
- semantic prior addresses

FITTED STATE

- factor/group coordinates
- centers, scales, knots, spline bases
- GP/HS GP domain and frequencies
- structured-term field shapes
- replay policy and dependency provenance

`prepared expression = (callable object, positional args, keyword args, axis) prepared term = (callable, source, fitted state, dependencies) prepared observation= (lhs, distribution, evidence, weights, row shape)`

Generated backends: same intent, different geometry

COLLABORATOR APPENDIX C

StanBlocks → Stan

```
1 parameters {  
2   vector[K] beta_pop;  
3   real<lower=0> sigma;  
4 }  
5 model {  
6   beta_pop ~ normal(beta_loc, beta_scale);  
7   y ~ normal_id_glm(X, 0, beta_pop, sigma);  
8 }
```

Compiler-visible blocks, activity analysis, generated quantities, BridgeStan coordinates.

Turing → DynamicPPL

```
1 @model function generated(plan)  
2   beta_pop ~ product_distribution(priors)  
3   mu = plan.design.matrix * beta_pop  
4   for i in eachindex(y)  
5     y[i] ~ Normal(mu[i], sigma)  
6   end  
7 end
```

Native callables, submodels, DynamicPPL conditioning/prediction, Julia AD.

Extension seams: where a contributor works

COLLABORATOR APPENDIX D

CHANGE	SHARED LAYER	BACKEND WORK
Callable distribution	Shape trait; dependency and axis validation.	Turing often executes directly; StanBlocks adds full-call translation, density, RNG, pointwise paths.
Structured term	Typed fitted state, fields, priors, replay.	SLIC emitter and/or native Julia submodel/effect method.
Evidence wrapper	Materialized bounds/endpoints and validation.	Executable CDF/density/RNG composition per backend.
Output	Logical identity and row/group segmentation.	Physical carrier discovery and regeneration.

CONTRIBUTOR RULE Add the semantic record once; let unsupported backends refuse by dispatch. Do not claim support until density, prediction, replay, and named-output behavior are tested at the intended boundary.

Boundaries are a product feature

COLLABORATOR APPENDIX E

KNOWN GAPS

- no universal feature cross-product
- unequal output/transport surfaces
- adaptive geometry not general across terms/backends
- structured kernels need explicit shape/AD contracts
- several replay geometries remain fail-closed

REQUIRED EVIDENCE

- construction diagnostics
- Stan compile or native model execution
- normalized density + mapped gradients
- predictive and pointwise semantics
- frozen replay and new-group behavior

STILL RESEARCH

- online geometry selection
- cross-backend equivalence criteria
- automatic marginalization
- mixed discrete/continuous structured inference
- fit portability across backend evolution

Failing loudly is part of parity: an unsupported model must not quietly acquire a different meaning.

Reproduce, inspect, extend

COLLABORATOR APPENDIX F

REPOSITORY TRAIL

- 1. docs/refactoring.md — architecture and fitted preparation
- 2. docs/src/feature-atlas.md — executable four-pane examples
- 3. docs/src/turing-backend.md — parity and boundaries
- 4. docs/backend_comparisons.jl — generated evidence surface
- 5. test/backend_comparisons.jl and focused backend tests
- 6. research/adaptive_centering/ — case-study provenance and harness

PRIMARY COMPARISON SOURCES

- Bürkner (2018), *Advanced Bayesian Multilevel Modeling with brms*
- current brms brm, brmsformula, custom_family, stanvar, and gp documentation
- Riutort-Mayol et al. (2023), practical HSGP approximation
- Papaspiliopoulos, Roberts & Sköld (2007), centered/noncentered framework
- Gorinova et al. (2020), automatic reparameterisation
- Stan User’s Guide, reparameterization chapter

Full URLs, immutable revisions, rendering commands, and claim-tier definitions are in the deck README. Prospective guests and cited authors have not reviewed or endorsed this material.